ACCESO A DATOS – 2º DAM

UP 2 – Bases de Datos Relacionales - JDBC

SESIÓN 1 - INSTALACIÓN DEL ENTORNO, GESTOR DE DEPENDENCIAS MAVEN Y ARQUITECTURA BASE

OBJETIVO

Antes de empezar con el tema, hemos de asegurarnos que tenemos todo lo necesario para trabajar instalado. Con el propósito de garantizar un correcto despliegue técnico para el desarrollo del proyecto práctico de la unidad en «Mutxamel Sec-Ops S.L.», realizaremos una caracterización técnica previa de cada componente que conforma nuestra infraestructura de trabajo y habilitaremos las bases de datos corporativas.

1. Instalación y Configuración del Entorno

Antes de empezar con el tema, hemos de asegurarnos de que tenemos todo lo necesario para trabajar instalado. Con el propósito de garantizar un correcto despliegue técnico para el desarrollo del proyecto práctico de la unidad en «Mutxamel Sec-Ops S.L.», realizaremos una caracterización técnica previa de cada componente que conforma nuestra infraestructura de trabajo.

1.1. Conceptos clave

Concepto / Herramienta	Definición y Propósito Funcional
Java JDK *(Java Development Kit)*	Kit de desarrollo oficial de Java. Proporciona el compilador (javac) para traducir el código fuente a bytecode y las herramientas de ejecución de la máquina virtual (JVM). (*Nota: Es indispensable frente a un JRE básico, ya que Maven y los procesos de compilación del proyecto requieren el JDK completo*).
IntelliJ IDEA	Entorno de Desarrollo Integrado (IDE). Plataforma de software principal para la codificación, gestión de dependencias mediante Maven y depuración (*debugging*) del proyecto.
SGBDR *(Sistema Gestor de Bases de Datos Relacionales)*	Software de infraestructura encargado del almacenamiento estructurado, control de transacciones, concurrencia y consulta de datos mediante el estándar SQL.
PostgreSQL y pgAdmin 4	PostgreSQL es nuestro SGBDR principal de código abierto para las prácticas de la unidad. pgAdmin 4 es la herramienta de administración visual (interfaz gráfica) para la gestión de esquemas, tablas y sentencias SQL del proyecto.

1.2. ¿Qué software se debe Instalar? (Requisitos de Infraestructura)

Para poder trabajar en esta unidad, asegúrate de descargar e instalar las siguientes herramientas oficiales en tu sistema operativo:

- **Kit de Desarrollo Java (JDK):** OpenJDK 21 LTS o superior.
- **Entorno de Desarrollo:** IntelliJ IDEA (Community o Ultimate Edition).
- **Servidor SGBDR:** PostgreSQL (versión 14 o 15) junto con su interfaz gráfica de administración pgAdmin 4.

1.3. Guía Paso a Paso de Instalación y Puesta en Marcha

Paso 1: Instalación de PostgreSQL y pgAdmin 4

1. **Descarga:** Accede al sitio web oficial de PostgreSQL y descarga el instalador oficial para la versión 14 o 15.
2. **Instalación:** Ejecuta el asistente marcando los componentes principales: **PostgreSQL Server**, **pgAdmin 4** y **Command Line Tools**.
3. **Configuración de Credenciales:** Durante el proceso, el instalador te pedirá una contraseña para el superusuario por defecto (postgres). Asigna una contraseña segura (por ejemplo, 123456 para las prácticas de aula) y mantén el **puerto por defecto (5432)**.
4. **Verificación:** Abre la aplicación **pgAdmin 4** instalada en tu equipo, introduce tu contraseña maestra y comprueba que el servidor local responde correctamente.

Paso 2: Verificación

Una vez finalizadas ambas instalaciones, debemos comprobar que los demonios de los motores de bases de datos están configurados para arrancar correctamente:

- Pulsa la combinación de teclas Win + R, escribe services.msc y pulsa Enter.
- Localiza el servicio de PostgreSQL (aparecerá como postgresql-x64-14 o 15) y comprueba que su estado es **«En ejecución»** (*Running*).

2. Creación de la Base de Datos

Una vez comprobado que el servicio de PostgreSQL está activo, vamos a crear la DB que utilizará Mutxamel Sec-Ops S.L. para registrar a los analistas de seguridad y las aplicaciones auditadas.

2.1. ¿Qué va a hacer nuestra aplicación (el proyecto)?

Para que tengas una visión global desde el primer día, la aplicación Java que vas a programar en esta unidad se conectará a esta base de datos para cumplir dos objetivos de negocio en la empresa de ciberseguridad:

1. **Control de Acceso (login):** Validar qué analistas de seguridad de Mutxamel Sec-Ops tienen permiso para acceder a la herramienta de auditoría de código.

2. **Inventario de Riesgos (software_auditado):** Consultar, listar y auditar el nivel de exposición al riesgo de las diferentes aplicaciones y portales web analizados (como el portal del Ayuntamiento o pasarelas de pago).

2.2. Creación de la Base de Datos en pgAdmin 4

Para simplificar la configuración y evitar problemas de permisos en clase, utilizaremos directamente el superusuario por defecto que crea PostgreSQL (postgres).

- **Paso 1:** Abre **pgAdmin 4**, despliega tu servidor local en el panel izquierdo (*Object Explorer*) e introduce la contraseña de administrador de PostgreSQL.
- **Paso 2:** Haz clic derecho sobre Databases -> Create -> Database... y crea una nueva base de datos llamada **mutxamel_pg** (puedes dejar a postgres como propietario por defecto).
- **Paso 3:** Haz clic derecho sobre tu nueva base de datos mutxamel_pg, abre la herramienta **Query Tool** y ejecuta el siguiente script SQL para crear las tablas y poblar los datos iniciales:

```
-- 1. Tabla 'login': Almacena las credenciales de los analistas

CREATE TABLE login (
    id SERIAL PRIMARY KEY,
    user_name VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

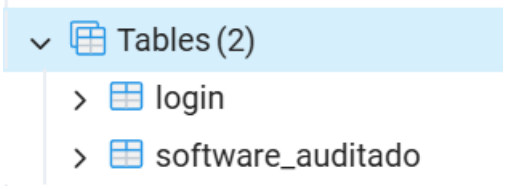
INSERT INTO login (user_name, password) VALUES
('cpgrau', 'sha256_9f86d081884c7d659a2feaa0c55ad015'),
('silvia_sec', 'sha256_4b227777d4dd1fc61c6f884f48641d02'),
('pentester_mutx', 'sha256_e3b0c44298fc1c149afbf4c8996fb924');

-- 2. Tabla 'software_auditado': Apps analizadas por la consultora

CREATE TABLE software_auditado (
    codigo SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    nivel_exposicion INTEGER NOT NULL
);

INSERT INTO software_auditado (nombre, nivel_exposicion) VALUES
('Ayuntamiento-Mutxamel-Portal', 3),
('App-Reserva-Instalaciones', 4),
('Pasarela-Pagos-Municipal', 5),
('Gestor-Expedientes-Urbanismo', 3);
```

Dentro de Schemas -> Tables encontrarás las tablas credas:



2.3. Exportación del Backup

Haz clic derecho sobre 'mutxamel_pg' -> 'Backup...' en formato Custom y guárdalo como 'back_mutxamel_s1.backup'.

Realizamos el *backup* para garantizar la portabilidad y una recuperación rápida del entorno ante cualquier error de programación. Además, nos proporciona una evidencia formal lista para su entrega y evaluación oficial en la plataforma Aules / GitHub.

3. Creación del Proyecto con Apache Maven y Driver de PostgreSQL

Una vez que ya tenemos nuestra base de datos 'mutxamel_pg' preparada en PostgreSQL, el siguiente paso es inicializar nuestro entorno de desarrollo en Java utilizando Apache Maven como gestor de dependencias.

3.1. Inicialización del Proyecto en IntelliJ IDEA

1. Abre IntelliJ IDEA y selecciona 'File -> New -> Project...'.
2. Elige Maven (o Maven Archetype seleccionando el arquetipo 'org.apache.maven.archetypes:maven-archetype-quickstart').
3. Comprueba que el JDK seleccionado es un entorno de desarrollo completo (OpenJDK 21 o superior).
4. En las opciones avanzadas (Advanced Settings), introduce las coordenadas corporativas del proyecto:
    - Name (Nombre del proyecto): mutxamelUP2
    - GroupId: com.mutxamel.secops
    - ArtifactId: mutxamelUP2
    - Version: 1.0-SNAPSHOT

3.2. Configuración del archivo `pom.xml`

Para evitar descargar archivos .jar manualmente, añade la dependencia oficial del driver JDBC de PostgreSQL dentro de la etiqueta <dependencies> de tu archivo pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.mutxamel.secops</groupId>
    <artifactId>mutxamelUP2</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>21</maven.compiler.source>
```

```
        <maven.compiler.target>21</maven.compiler.target>
    </properties>

    <dependencies>
        <!-- Dependencia para pruebas unitarias por defecto -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>3.8.1</version>
            <scope>test</scope>
        </dependency>

        <!-- Driver JDBC Oficial para PostgreSQL -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.7.4</version>
        </dependency>
    </dependencies>
</project>
```

Importante: Tras guardar el archivo, pulsa el botón de recarga (Load Maven Changes o clic derecho en el pom.xml -> Maven -> Reload project) para que Maven descargue automáticamente el driver a tu repositorio local .m2.

3.3. Conexión visual desde el panel Database de IntelliJ IDEA

Para visualizar las tablas y comprobar los datos de mutxamel_pg sin tener que cambiar constantemente a pgAdmin 4, podemos vincular PostgreSQL directamente en el entorno de desarrollo:

1. En el lateral derecho de IntelliJ IDEA, haz clic en el icono de la pestaña Database (DB).
2. Pulsa en el icono + -> Data Source -> PostgreSQL.
3. Introduce los parámetros de conexión: Host (localhost), Port (5432), Database (mutxamel_pg), User (postgres) y la contraseña que configuraste al instalar PostgreSQL.
4. Si en la parte inferior aparece un aviso indicando que falta el driver, pulsa en Download para que IntelliJ lo descargue automáticamente.
5. Haz clic en Test Connection para verificar que aparece el tic verde de conexión establecida y pulsa OK.

4. Estructura de Paquetes y Conexión con Patrón Singleton

4.1. ¿Qué es un patrón de diseño y por qué es importante?

Un patrón de diseño es una solución probada, estandarizada y reutilizable para resolver un problema habitual de arquitectura y desarrollo de software. No es un fragmento de código que se copia y pega, sino una pauta conceptual sobre cómo estructurar las clases y los componentes de una aplicación de forma limpia y ordenada.

Su uso es fundamental en el desarrollo profesional por tres razones clave:

- **Mantenibilidad**: El código se vuelve mucho más fácil de leer, corregir y ampliar en el futuro sin romper otras partes del sistema.
- **Separación de responsabilidades**: Cada clase hace una única cosa y la hace bien (Principio de Responsabilidad Única).
- **Escalabilidad y trabajo en equipo**: Permite que varios desarrolladores trabajen de forma paralela sin pisarse el código.

4.2. Patrones aplicados en nuestro proyecto (MVC / DAO / Singleton)

En «Mutxamel Sec-Ops S.L.» estructuraremos la carpeta `src/main/java de tu proyecto mutxamelUP2` creando el paquete base 'ud2' y dentro de él estos cuatro subpaquetes:

- **ud2.model (Modelo / POJOs):** Define las clases Java que representan las entidades del negocio, como Login. Sus atributos se corresponden exactamente con las columnas de las tablas de PostgreSQL.
- **ud2.dao (Data Access Object - DAO):** Es el núcleo del acceso a datos. Encapsula todas las sentencias SQL (SELECT, INSERT, etc.). La aplicación principal no se preocupa de cómo se conecta a la base de datos, sino que pide los objetos a clases como LoginAccessDB.
- **ud2.util (Utilidades y Patrón Singleton):** Gestiona recursos transversales como la conexión a la base de datos (DatabaseConnection). Aplicamos el Patrón Singleton para garantizar que solo exista una única conexión abierta en toda la aplicación, evitando saturar el servidor PostgreSQL.
- **ud2.app (Aplicación / Vista):** Es el punto de entrada (MainApp). Orquesta la ejecución del programa llamando a los DAO y mostrando los resultados.

4.2.1. Patrón Singleton

El **Patrón Singleton** (o patrón de instancia única) es un patrón de diseño creacional que garantiza que una clase tenga **una única instancia** en toda la aplicación y proporciona un punto de acceso global a la misma.

Características Clave

Para implementarlo (como vimos en las conexiones a bases de datos), se siguen tres reglas básicas:

- **Constructor privado (private):** Impide que cualquier otra clase pueda crear objetos utilizando el operador new.
- **Atributo estático:** Almacena la única referencia de la clase dentro de sí misma.
- **Método público estático (public static):** Funciona como el punto de acceso global que devuelve esa única instancia (creándola solo si todavía no existe).

¿Para qué se utiliza?

Es fundamental cuando se necesita coordinar acciones o controlar el acceso a recursos compartidos y costosos, tales como:

- **Conexiones a bases de datos:** Para evitar abrir canales innecesarios de red y reutilizar una única conexión activa.
- **Gestores de configuración o sistemas de registros (*logs*):** Donde un único punto centralizado debe administrar el estado global del sistema.

4.3. Código Base de los Componentes

Paquete ud2.util → DatabaseConnection.java (Patrón Singleton)

```
package ud2.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    private static Connection connection = null;

    private DatabaseConnection() {}

    static {
        String url = "jdbc:postgresql://localhost:5432/mutxamel_pg";
        String user = "postgres";
        String password = "tu_contraseña"; // Contraseña de tu PostgreSQL

        try {
            connection = DriverManager.getConnection(url, user, password);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public static Connection getConnection() {
        return connection;
    }
}
```

4.4. Prueba de Ejecución en MainApp

Creamos la clase principal de prueba para verificar que la conexión responde correctamente:

Paquete ud2.app -> MainApp.java

```
package ud2.app;

import ud2.util.DatabaseConnection;
import java.sql.Connection;
import java.sql.SQLException;


public class MainApp {
    public static void main(String[] args) {
        try (Connection con = DatabaseConnection.getConnection()) {
            if (con != null && !con.isClosed()) {
                System.out.println("BD conectada correctamente.");
            } else {
                System.out.println("No se ha podido establecer la conexión.");
            }
```

```
        } catch (SQLException e) {
            System.err.println("Error de conexión: " + e.getMessage());
        }
    }
}
```

5. Validación en clase

Para dar por superada esta primera sesión, avísame cuando acabes para mostrar en tu equipo los siguientes puntos funcionando:

1. Base de datos y Backup: La base de datos mutxamel_pg creada en PostgreSQL con las dos tablas pobladas (login y software_auditado) y el archivo back_mutxamel_s1.backup guardado dentro de una carpeta /backup/ en el proyecto.

2. Estructura y Maven: El proyecto mutxamelUP2 creado con sus cuatro paquetes (ud2.util, ud2.model, ud2.dao, ud2.app) y el driver de PostgreSQL cargado sin errores en el pom.xml.

3. Panel Database: La conexión configurada y activa en la pestaña lateral Database de IntelliJ IDEA.

4. Ejecución en directo: Ejecutar MainApp.java y mostrar por consola el mensaje "BD conectada correctamente.".